

虚拟艺术画廊展示系统说明书

一、系统需求分析

1.1 系统功能概述

本虚拟艺术画廊展示系统旨在为用户提供一个沉浸式、交互式的在线艺术欣赏平台。系统通过现代化的 Web 技术，模拟了真实画廊的浏览体验，并加入了丰富的个性化与社交功能。系统主要功能如下：

- 画廊浏览与探索：
 - 2D 漫游式画廊 (GalleryView1.vue)：用户可以通过鼠标拖拽和滚轮缩放，在宏大的 2D 宇宙空间中自由漫游，探索以瀑布流形式随机分布的艺术作品，点击作品可以跳转详情页。
 - 3D 行星式画廊 (GalleryView.vue)：作为项目的加分项实现，用户可以在一个以行星为中心的 3D 宇宙中，通过旋转和缩放视角来浏览环绕在行星周围的艺术作品，并可以通过点击环绕行星的时间轴定位相关作品。
 - 风格筛选：在两种画廊模式下，用户均可根据“古典”、“现代”、“抽象”等艺术风格筛选和聚焦特定类型的作品。
 - 艺术时间轴：在 2D 画廊中，页面底部设有艺术时间轴，标注了关键的艺术年份，用户点击年份可快速定位到该时期的作品区域。
- 画作深度展示：
 - 详细信息页 (ArtworkDetail.vue)：用户点击任意画作可进入独立的详情页面，查看作品的高清大图、标题、作者、创作年代、艺术风格、作品描述和背景

故事等。

- 放大镜功能：在画作详情页，提供悬停放大镜和全屏放大两种模式，方便用户观察作品的笔触和细节。
- 背景音乐：每幅画作均配有专属的背景音乐，用户可以控制播放、暂停、调节音量和查看进度。

- 个人画廊定制 (PersonalGallery.vue):

- 创建与编辑：注册用户可以创建和编辑属于自己的个人画廊，自定义画廊名称、描述、主题背景和作品排列方式。
- 作品收藏与布局：用户可以从系统提供的所有画作中，通过拖拽方式将喜爱的作品添加至个人画廊的“工作区”，并自由调整作品的位置、大小和旋转角度。
- 权限管理：支持将画廊设置为“公开”、“好友可见”或“私密”三种不同的访问权限。

- 社区与互动体验：

- 核心互动 (ArtworkDetail.vue)：用户可以对画作进行点赞和发表评论。
- 画廊社区 (GalleryCommunity.vue)：提供一个完整的社交空间，用户可以搜索并添加好友、处理好友请求、与好友进行实时聊天。
- 分享功能：用户可以将自己创建的画廊或喜爱的作品通过链接分享给他人。

在社区中，可以直接将画廊分享给指定好友。

- 用户认证系统 (LoginView.vue, RegisterView.vue):

- 提供完整的用户注册和登录功能，通过 JWT (Token)进行身份验证，并利用前端路由守卫保护需要登录才能访问的页面。

1.2 系统用例图

系统包含“游客”和“注册用户”两种角色。注册用户继承游客的所有功能，并拥有额外的个性化与社交权限。系统的核心用例关系如图 1-1 所示。

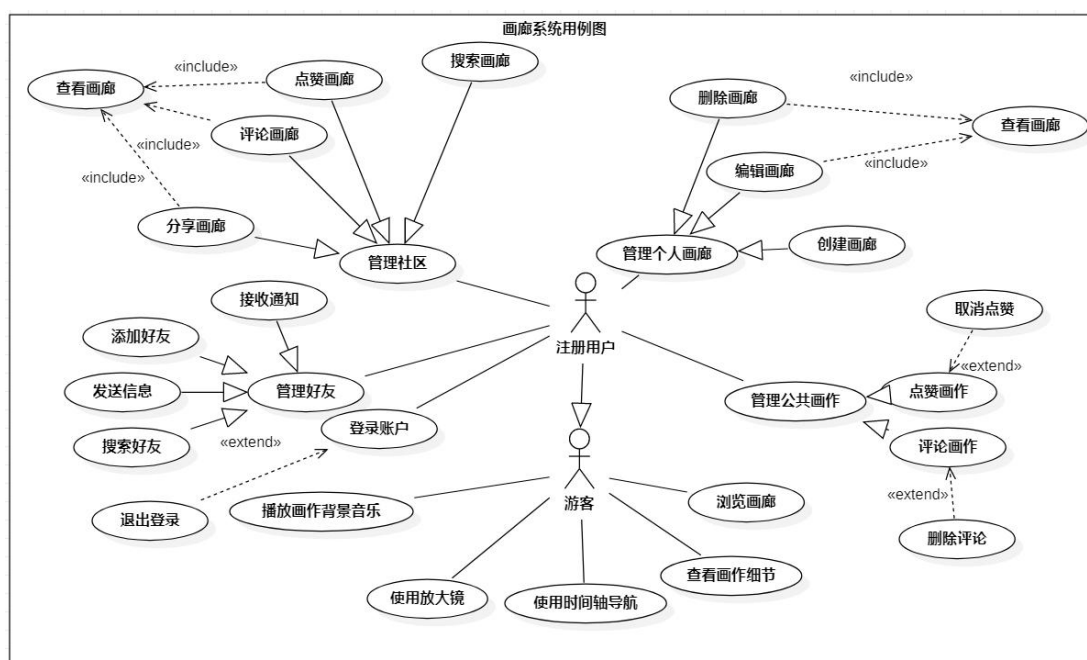


图 1-1 用例图

1.3 系统功能结构图

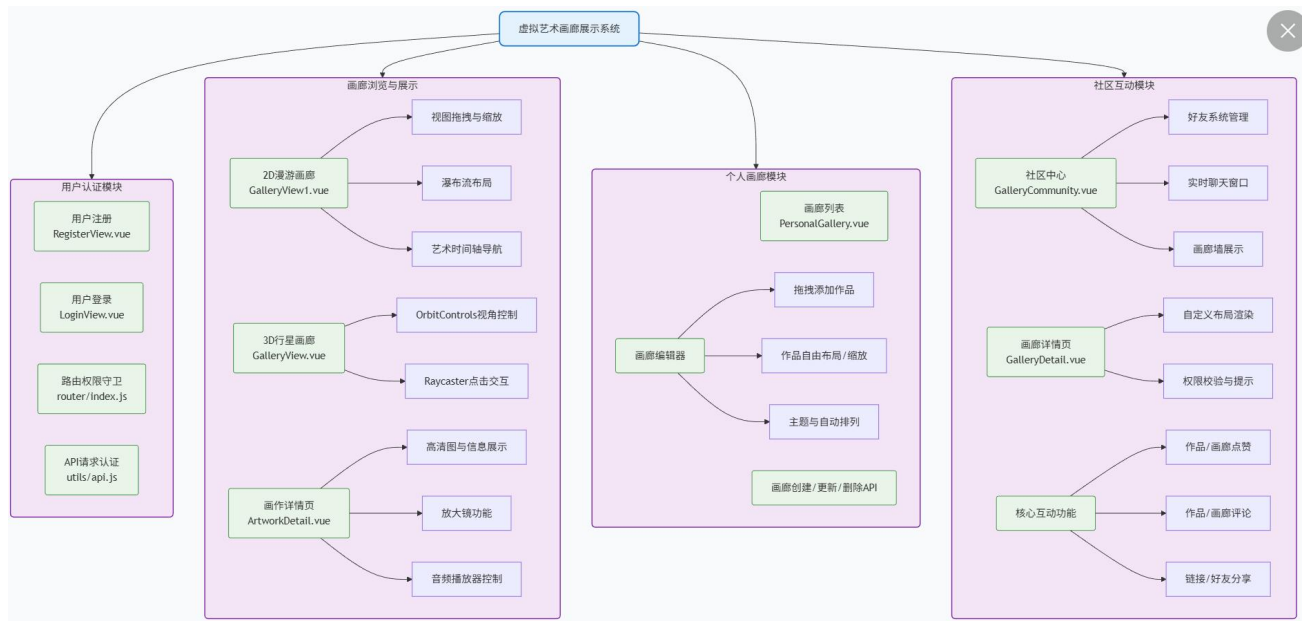


图 1-2 功能结构图

二、系统设计

2.1 前端架构设计

本系统采用现代前端技术栈进行构建，整体架构清晰、模块化、易于维护和扩展。

- **组件化开发 (Vue.js):** 系统基于 Vue 3 Options API 和 `<script setup>` 语法糖进行开发。整个应用被拆分为多个高内聚、低耦合的组件。例如，`HomeView.vue` 负责首页展示，`GalleryView1.vue` 负责 2D 画廊，`ArtworkDetail.vue` 负责画作详情，`PersonalGallery.vue` 负责个人画廊管理，而 `GalleryCommunity.vue` 则集成了复杂的社区功能。这种方式使得代码复用性高，逻辑清晰。
- **单页应用导航 (Vue Router):** 系统使用 `vue-router` 实现单页应用 (SPA) 的路由管理。在 `router/index.js` 文件中，我们定义了应用的全部路径，如主页 `/`、

画廊 /gallery1、画作详情 /artwork/:id、个人画廊 /personal-gallery 和社区 /gallery-community 等。特别地，我们利用路由的 props: true 功能，将动态路由参数（如作品 ID）直接传递给组件。同时，通过 beforeEach 全局导航守卫，实现了对需要登录权限的路由（如 /personal-gallery）的访问控制，未登录用户会被重定向至登录页。

- **集中式状态管理 (Vuex):** 为了有效管理整个应用的共享状态，系统引入了 Vuex。在 store/index.js 文件中，定义了核心的状态 (state)，如用户信息 user、登录状态 isLoggedIn、所有画作数据 artworks、个人画廊列表 personalGalleries 以及社区相关数据（好友、消息等）。通过 mutations 来同步修改状态，actions 来处理异步操作（如 API 请求），getters 来派生状态。例如，用户登录后，会调用 login action，该 action 会提交 SET_USER mutation 来更新用户状态，并触发 initializeCommunityData action 来异步获取该用户的社区数据。

2.2 画廊布局设计

为了提供多样化的浏览体验，系统设计了两种画廊布局模式。

- **2D 漫游式画廊 (GalleryView1.vue):** 此视图的核心是利用 <canvas> 元素和 Canvas API 绘制动态的星空和网格背景，模拟出无限延伸的宇宙空间感。画作本身则是普通的 <div> DOM 元素，通过 CSS 的 position: absolute 和 transform 属性进行定位。用户的浏览操作通过监听鼠标事件实现：

- **拖拽移动：**通过监听 `mousedown`, `mousemove`, `mouseup` 事件，在 `drag` 方法中计算鼠标位移 `dx`, `dy`，并更新组件数据 `offset.x` 和 `offset.y`。这个 `offset` 值被应用到所有画作和背景的变换中，从而实现整个画布的平移效果。
- **滚轮缩放：**通过监听 `wheel` 事件，在 `handleWheel` 方法中计算缩放比例，并更新 `scale` 值。为了实现以鼠标为中心的缩放，该方法还根据鼠标当前位置对 `offset` 进行补偿计算。
- **画作详情展示 (ArtworkDetail.vue)：**当用户点击画作时，会跳转到此页面。
 - **放大镜：**通过监听画作图片的 `mousemove` 事件，在 `handleZoom` 方法中实时获取鼠标坐标，并计算 `backgroundPosition`。这个样式被应用到一个独立的 `zoom-lens-global` 元素上，该元素跟随鼠标移动，从而实现了局部放大的效果。用户还可以通过滑块调整 `zoomLevel`。
 - **背景音乐：**每个画作关联一个音频文件。`toggleAudio` 方法负责初始化 `Audio` 对象并控制播放与暂停。播放进度、音量调节等均通过操作 `Audio` 对象并更新 `Vue` 的响应式数据来实现。

2.3 样式与界面设计

系统整体采用“星空宇宙”的美学风格，旨在营造沉浸、静谧而富有科技感的艺术氛围。

- **统一视觉主题：**在多个组件（如 `HomeView`, `ArtworkDetail`, `LoginView`）中，

都复用了 `.starry-background` 和 `.meteor-shower` 等 CSS 类，通过动态生成星星和流星的 DOM 元素并用 CSS 动画驱动，确保了视觉风格的统一性。

- **毛玻璃与光晕效果：**广泛使用 `backdrop-filter: blur()` 属性创建磨砂玻璃质感，如 `frosted-overlay` 和各种卡片 `cosmic-card`。同时，利用 `box-shadow` 和 `filter: drop-shadow()` 为标题、按钮和高亮元素添加荧光 (glow) 效果，增强了科技感和视觉焦点。
- **响应式布局：**在 `PersonalGallery.css` 和 `GalleryView1.css` 等文件中，大量使用了 `@media` 查询来适应不同尺寸的屏幕。例如，在窄屏幕下，画廊网格 `.galleries-grid` 的列数会自动调整，编辑器的布局也会从三列变为垂直堆叠，保证了在移动设备上依然有良好的使用体验。
- **复古画框设计：**为了突出艺术作品的质感，在 `ArtworkDetail.vue` 和 `PersonalGallery.vue` 等组件中，通过多个 `<div>` 元素和 CSS 样式，精心模拟了具有立体感和纹理的木质画框 (`.wooden-frame`)，增强了作品的仪式感。

三、系统实现

3.1 系统开发环境

开发环境：Node.js、Vue CLI

技术栈：Vue.js + Vue Router + Vuex + Canvas API

兼容性：支持主流浏览器 (Chrome、Firefox、Edge)

3.2 核心功能实现

3.2.1 首页 (HomeView.vue)

- 功能描述：作为应用的门户和引导页，首页首先展示一个带有“进入星空画廊”按钮的动态入口，吸引用户进入。当用户激活画廊后，页面会平滑滚动到一个横向滚动的精选博物馆作品展示区。
- 关键实现：
 1. 入口过渡：通过 galleryActive 这一布尔值数据，利用 `:class="{ 'portal-hidden': galleryActive }"` 为入口界面添加 CSS 类，实现其淡出和主内容区的激活效果。activateGallery 方法会触发页面平滑滚动到画廊展示区。
 2. 动态星空背景：在 mounted 生命周期钩子中调用 initStars 方法，动态创建大量 div.star 元素，每个星星的位置、大小和闪烁动画（twinkle）的延迟与周期都是随机生成的，创造了逼真的星空效果。
 3. 横向滚动画廊：核心是一个名为 .gallery-track 的 div，其宽度通过 trackStyle 计算属性动态设置为所有作品宽度之和。通过改变 transform: translateX(-this.scrollPosition) 属性来实现滚动。scrollToArtwork 方法使用 requestAnimationFrame 创建平滑的滚动动画，而非瞬时跳转。
 4. 交互与状态：activeArtworkId 记录当前居中的作品 ID，selectedArtwork 存储当前点击查看的作品详情。点击作品会调用 selectArtwork 更新这两个状态，并触发滚动。

关键截图：

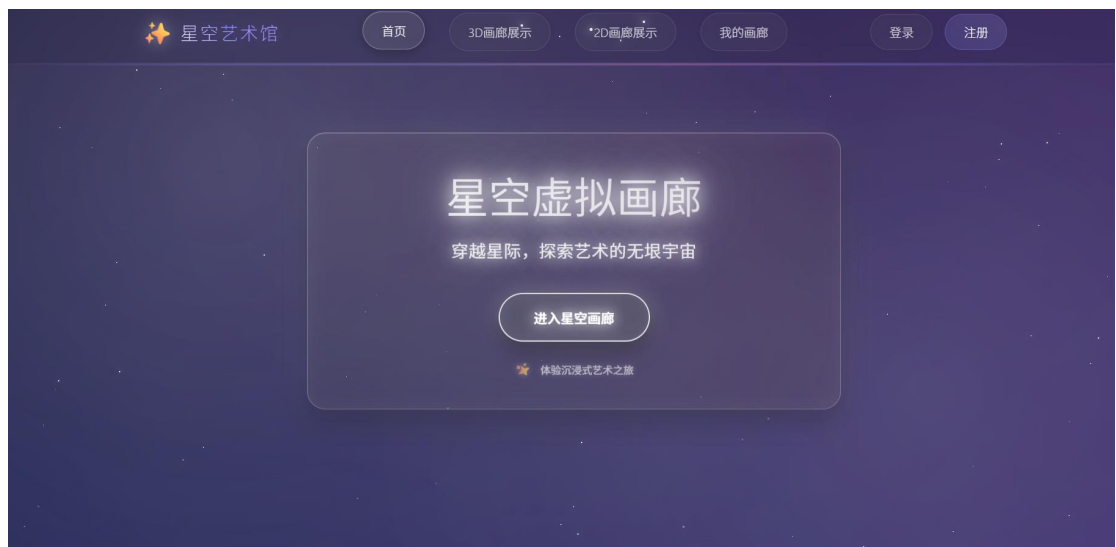


图 3.1 主页门户框

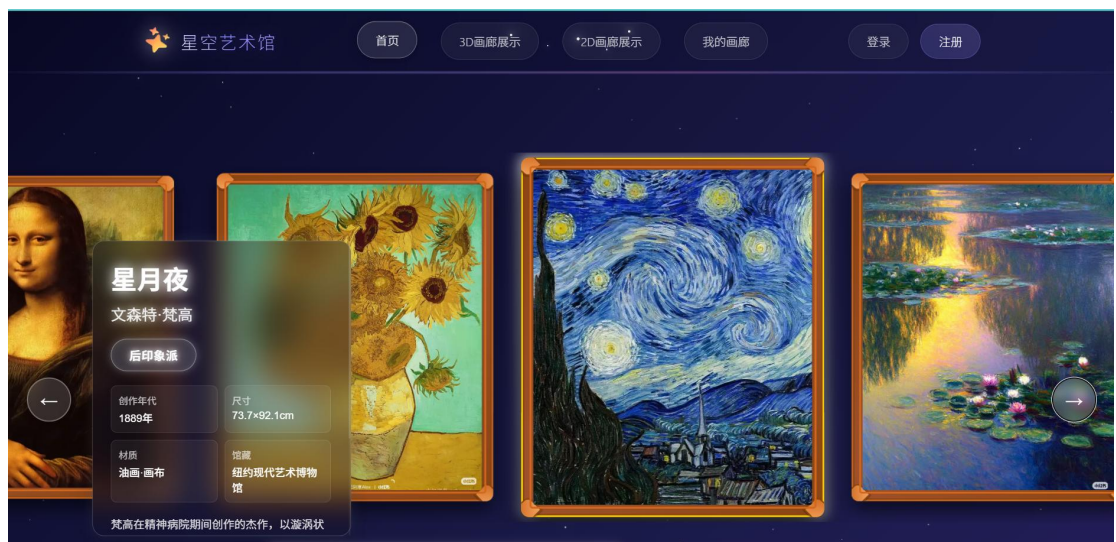


图 3.2 页面滚动到横向画廊

关键代码：

```
309 | | },
310 | | selectArtwork(artwork, event) {
311 | |   event.stopPropagation();
312 | |   this.activeArtworkId = artwork.id
313 | |   this.selectedArtwork = artwork
314 | |   this.scrollToArtwork(artwork.id)
315 | | },
316 | | scrollToPrevArtwork() {
317 | |   const currentIndex = this.museumArtworks.findIndex(art => art.id === this.activeArtworkId);
318 | |   const prevIndex = currentIndex > 0 ? currentIndex - 1 : this.museumArtworks.length - 1;
319 | |   this.selectArtwork(this.museumArtworks[prevIndex], { stopPropagation: () => {} });
320 | | },
321 | | scrollToNextArtwork() {
322 | |   const currentIndex = this.museumArtworks.findIndex(art => art.id === this.activeArtworkId);
323 | |   const nextIndex = currentIndex < this.museumArtworks.length - 1 ? currentIndex + 1 : 0;
324 | |   this.selectArtwork(this.museumArtworks[nextIndex], { stopPropagation: () => {} });
325 | | },
326 | | scrollToArtwork(artworkId) {
327 | |   const index = this.museumArtworks.findIndex(art => art.id === artworkId)
328 | |   const artworkWidth = 380
329 | |   const gap = 50
330 | |   const containerWidth = this.$refs.galleryContainer ? this.$refs.galleryContainer.offsetWidth : 1200
331 | |   const targetPosition = index * (artworkWidth + gap) - (containerWidth - artworkWidth) / 2
332 | |
333 | |   if (this.scrollAnimation) {
```

图 3.3 画廊滚动实现的逻辑

3.2.2 用户认证流程

- 功能描述：提供用户注册和登录功能，是进入系统个性化功能（如个人画廊、社区）的入口。界面风格与首页保持一致，营造统一的宇宙主题感。
- 关键实现：
 1. 注册 (RegisterView.vue)：用户填写表单后，handleRegister 方法对密码一致性等进行前端校验，然后调用 authAPI.register 发送请求。成功后，后端返回 token 和用户信息，前端将其存入 localStorage，并通过 Vuex 的 login action 更新全局状态，最后跳转到首页。
 2. 登录 (LoginView.vue)：handleLogin 方法调用 authAPI.login。成功后执行与注册相同的本地存储和状态更新流程。
 3. 身份维持：应用启动时，Vuex store 会从 localStorage 初始化 user 和 isLoggedIn 状态，实现登录状态的持久化。

4. API 请求认证 (api.js): 使用 axios 拦截器。apiWithAuth 实例在每次请求前, 都会从 localStorage 读取 token 并添加到 Authorization 请求头中。
5. 表单处理: 使用 <form> 标签包裹输入框, 并通过 v-model 将输入内容双向绑定到 data 中的 form 对象。通过 @submit.prevent 修饰符监听表单提交事件, 并调用 handleLogin 或 handleRegister 方法。
6. 状态管理与持久化: 认证成功后, 后端返回的用户信息和 token 会被 localStorage.setItem 存入浏览器本地存储, 以实现持久化登录。同时, 调用 Vuex 的 store.dispatch('login', response.user) 方法, 将用户信息存入全局状态, 并更新 isLoggedIn 标志。
7. 路由跳转: 登录成功后, 使用 this.router.push(redirect) 进行页面跳转。
redirect 值来源于路由查询参数, 确保用户能返回到登录前尝试访问的页面。

关键截图:



图 3.4 注册页面

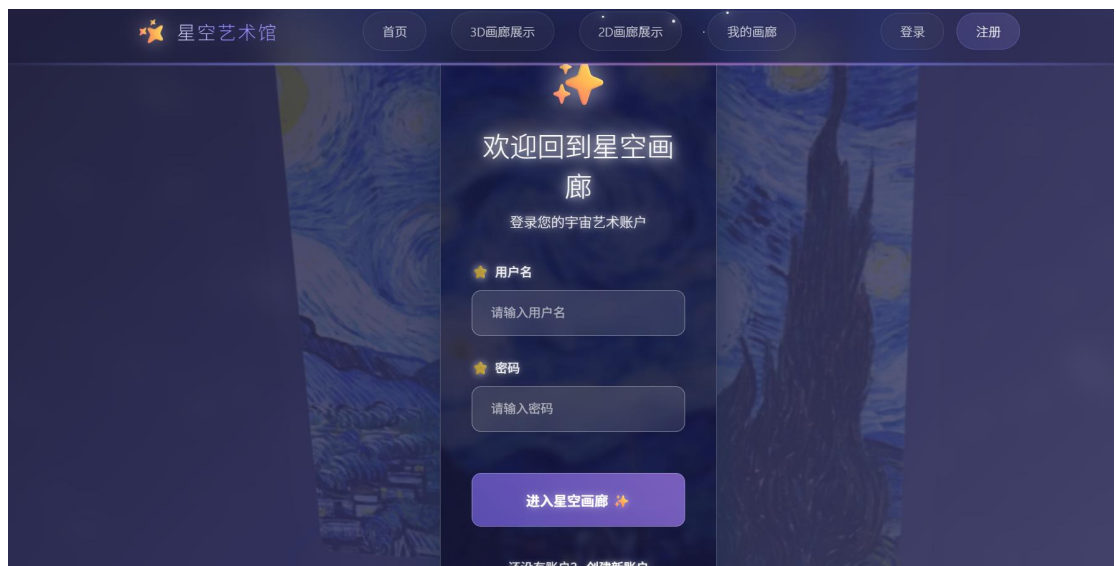


图 3.5 登录页面

代码实现：

```
src > router > JS index.js > routes
121
122 // 路由守卫
123 router.beforeEach((to, from, next) => {
124   // 设置页面标题
125   if (to.meta.title) {
126     document.title = to.meta.title
127   }
128
129   // 检查是否需要认证
130   const requiresAuth = to.matched.some(record => record.meta.requiresAuth)
131
132   // 从localStorage获取登录状态
133   const user = localStorage.getItem('user')
134   const isLoggedIn = !!user
135
136   if (requiresAuth && !isLoggedIn) {
137     // 需要登录但未登录, 跳转到登录页
138     next({
139       name: 'Login',
140       query: {
141         redirect: to.fullPath,
142         message: '请先登录以访问个人画廊'
143       }
144     })
145   } else if ((to.name === 'Login' || to.name === 'Register') && isLoggedIn) {
146     // 已登录但访问登录/注册页, 跳转到首页
147     next({ name: 'Home' })
148   } else {
149     // 正常导航
150     next()
151   }
152 })
153
```

图 3.6 登录跳转逻辑

3.2.3 2D 画廊浏览与时间轴

- 功能描述：提供一个广阔的、可自由探索的 2D 艺术空间。用户可以通过鼠标平移和缩放视图，画作以瀑布流形式分布在空间中。页面底部提供了一个可交互的时间轴。

- 关键实现：

1. 视图实现 (GalleryView1.vue): 背景由 `<canvas>` 绘制, `drawBackground` 方法负责绘制渐变背景和网格。画作是绝对定位的 `div.artwork-item`。
`getArtworkStyle` 方法根据 `offset` 和 `scale` 计算每个画作的 `transform` 样式, 实现视图的移动和缩放。
2. 瀑布流布局 (`masonryArtworks` 计算属性): 通过一个简单的算法, 动态计算每个画作在多列布局中的 `x` 和 `y` 坐标, 确保画作在空间中分布自然且不会重叠。
3. 时间轴交互: `timelineEras` 计算属性从所有画作中提取出关键年份, 并计算其在时间轴上的百分比位置。当用户点击某个年份时, `scrollToEra` 方法会找到该年份对应的第一幅画作, 并计算出一个目标 `offset`, 最后通过 `animateToOffset` 方法平滑地将视图中心移动到该画作的位置。
4. 混合渲染技术: 背景是一个 `<canvas>` 元素, 利用 `Canvas` API 在 `drawBackground` 方法中绘制动态的网格和星云等视觉效果。而画作本身是普通的 `div` 元素, 通过 `v-for` 渲染。这种混合渲染的方式既保证了背景的动态性, 又利用了 `DOM` 和 `CSS` 在处理画作交互和样式上的便利。
5. 无限视图控制:
 - (1) 平移: 通过监听 `mousedown`、`mousemove` 和 `mouseup` 事件, 在 `drag` 方法中实时更新 `offset` 数据, 从而改变所有画作的 `transform: translate(...)` 样式, 实现视图的平移。
 - (2) 缩放: 通过监听 `wheel` 事件, 在 `handleWheel` 方法中修改 `scale` 数据。
为了实现“以鼠标为中心缩放”, 该方法还根据鼠标在画布上的坐标

对 offset 进行了补偿计算。

关键截图：



图 3.7 2D 画廊

关键代码：

```
771 |  
772 | handlewheel(event) {  
773 |   event.preventDefault()  
774 |  
775 |   const delta = -Math.sign(event.deltaY) * 0.1  
776 |   const newScale = Math.max(0.3, Math.min(3, this.scale + delta))  
777 |  
778 |   const rect = this.$refs.canvas.getBoundingClientRect()  
779 |   const mouseX = event.clientX - rect.left  
780 |   const mouseY = event.clientY - rect.top  
781 |  
782 |   const scaleFactor = newScale / this.scale  
783 |  
784 |   this.offset.x = mouseX - (mouseX - this.offset.x) * scaleFactor  
785 |   this.offset.y = mouseY - (mouseY - this.offset.y) * scaleFactor  
786 |  
787 |   this.scale = newScale  
788 | },  
789 |
```

图 3.8 拖拽实现逻辑

3.2.4 3D 行星画廊 (加分项, GalleryView.vue)

- 功能描述：这是一个基于 Three.js 实现的 3D 虚拟画廊。艺术作品环绕着一个中央行星或几何体陈列，用户可以通过鼠标进行 3D 旋转、平移和缩放，提供 stronger 的沉浸感。
- 关键实现：
 1. 场景初始化：在 Mounted 钩子中调用 initThree 方法，完成 THREE.Scene、PerspectiveCamera、WebGLRenderer 的创建。使用 OrbitControls 插件，让用户能方便地控制视角。
 2. 动态对象创建：rebuildScene 方法负责根据用户选择的艺术风格（currentStyle）动态构建 3D 场景。createCentralObject 方法创建中央的星球（球体、立方体等），并根据风格将画作（作为带有贴图的 THREE.Mesh）布置在星球表面或环绕轨道上。
 3. 交互检测：通过 THREE.Raycaster 实现交互。在 onMouseMove 和 onCanvasClick 事件中，从相机和鼠标位置发射一条射线，检测与哪些 3D 对象相交。通过检查相交对象的 userData，判断用户是点击了画作还是时间轴标记，并触发相应操作。
 4. 镜头动画：当用户点击一个画作时，viewArtworkDetail 方法会启动一个镜头动画。它不直接改变相机位置，而是设置一个起始和结束位置/目标点，然后在 animate 渲染循环中，通过 requestAnimationFrame 和一个缓动函数 easeInOutCubic，在每一帧中平滑地插值相机的位置和朝向，实现电影般

的聚焦运镜效果。

关键截图：



图 3.9 3D 画廊

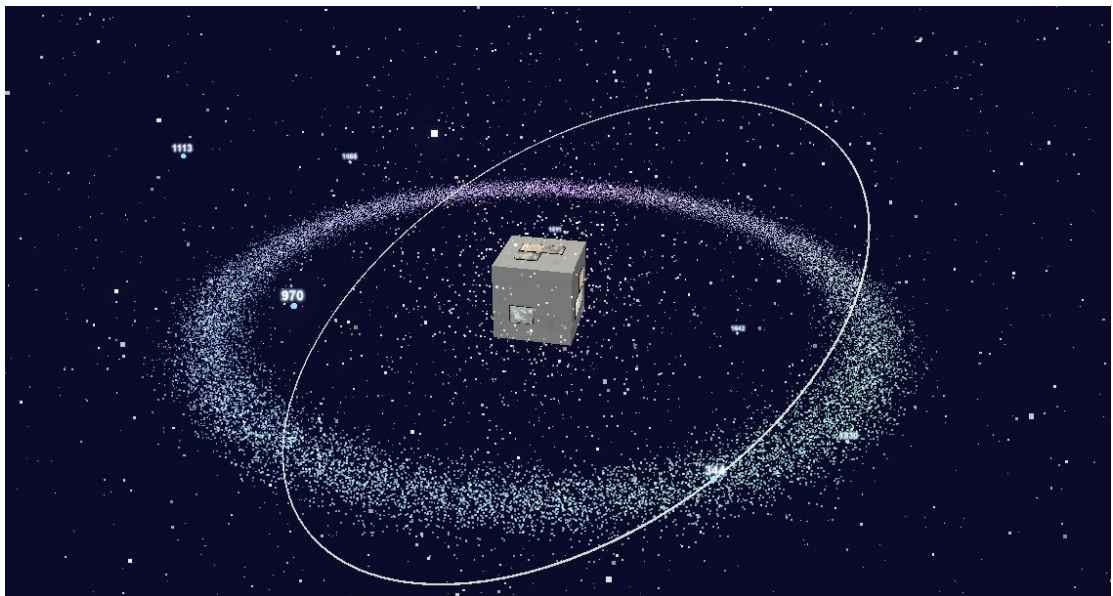


图 3.10 3D 画廊

关键代码：


```

'68 const focusOnYear = (year) => {
'69   const artworksOfYear = artworkGroupsForRaycast.value.filter(group => {
'70     return group.children.find(c => c.userData.artwork && c.userData.artwork.year === year);
'71   });
'72   if (artworksOfYear.length === 0) return;
'73   const centerPoint = new THREE.Vector3();
'74   artworksOfYear.forEach(group => {
'75     const worldPos = new THREE.Vector3();
'76     group.getWorldPosition(worldPos);
'77     centerPoint.add(worldPos);
'78   });
'79   centerPoint.divideScalar(artworksOfYear.length);
'80   const direction = new THREE.Vector3().subVectors(camera.value.position, centerPoint).normalize();
'81   const focusDistance = 60;
'82   cameraAnimation.value.active = true;
'83   cameraAnimation.value.startTime = Date.now();
'84   cameraAnimation.value.duration = 1500;
'85   cameraAnimation.value.startPos.copy(camera.value.position);
'86   cameraAnimation.value.endPos.copy(
'87     centerPoint.clone().add(direction.multiplyScalar(focusDistance))
'88   );
'89   cameraAnimation.value.startTarget.copy(controls.value.target);
'90   cameraAnimation.value.endTarget.copy(centerPoint);
'91   if (artworksOfYear.length > 0) {
'92     const firstArtwork = artworksOfYear[0].children.find(c => c.userData.artwork);
'93     if (firstArtwork) {
'94       selectedArtwork.value = firstArtwork.userData.artwork;
'95       if (autoRotate.value) toggleAutoRotate();
'96     }

```

图 3.11 时间轴定位画作逻辑实现

3.2.5 画作详情页 (ArtworkDetail.vue)

- 功能描述：作为艺术作品的最终展示终端，此页面集成了高清图、详细信息、交互式放大镜、背景音乐播放器和评论区。
- 关键实现：
 1. 数据加载与组合：在 mounted 钩子中调用 loadArtwork 方法。该方法首先从 Vuex 的 artworks 数组中获取基础数据，然后如果用户已登录，会进一步调用 artworkAPI.getDetail 和 commentAPI.getByArtwork 从后端获取最新的点赞状态和评论列表，实现了本地数据与远程数据的结合。
 2. 放大镜功能：通过监听画作图片的 mousemove 事件，在 handleZoom 方法中实时更新一个独立的放大镜 div（.zoom-lens-global）的 backgroundImage, backgroundPosition 和 left/top 样式，从而创建一个跟

随鼠标移动且内容同步放大的“放大镜”效果。

3. 自定义音频播放器：组件内有一个隐藏的 `<audio>` 元素，通过 `ref` 引用。
`initAudio` 方法为该元素绑定了 `loadedmetadata`、`timeupdate`、`play`、`pause` 等事件监听器，这些监听器会更新 Vue 组件的响应式数据（如 `duration`、`currentTime`），从而驱动 UI 上进度条和时间显示的更新。用户的点击操作（如播放、静音、拖动进度条）则反向调用 `audioElement` 的方法（如 `.play()`、`.pause()`、`.currentTime = ...`）来控制音频。
4. 评论功能：评论列表通过 `v-for` 渲染。`addComment` 方法调用 `commentAPI.create` 发送新评论到后端，成功后调用 `loadComments` 刷新列表。`handleDeleteComment` 方法实现了“乐观更新”，即先从前端 UI 中移除评论，再向后端发送删除请求。如果请求失败，它会将之前移除的评论重新插入，保证 UI 的响应速度和数据一致性。

关键截图：



图 3.12 作品详情页



图 3.13 背景音乐播放器

放大镜代码逻辑实现：

```
updateGlobalZoom(event) {  
  const imageEl = this.$refs.artworkImage  
  if (!imageEl || !this.isZoomActive) return  
  
  const rect = imageEl.getBoundingClientRect()  
  
  // 确保鼠标在画作范围内  
  if (  
    event.clientX < rect.left ||  
    event.clientX > rect.right ||  
    event.clientY < rect.top ||  
    event.clientY > rect.bottom  
  ) {  
    this.isZoomActive = false // 鼠标移出范围时也禁用放大镜  
    this.globalZoomStyle = {}  
    return  
  }  
}
```

```
// 如果之前是 false, 重新设为 true
if (!this.isZoomActive) {
  this.isZoomActive = true
}

// 计算相对位置
const x = (event.clientX - rect.left) / rect.width
const y = (event.clientY - rect.top) / rect.height

// 计算放大镜位置 (鼠标居中)
const lensSize = this.lensSize
const viewportWidth = window.innerWidth
const viewportHeight = window.innerHeight

let lensX = event.clientX - lensSize / 2
let lensY = event.clientY - lensSize / 2

// 边界检查, 防止放大镜超出视窗
lensX = Math.max(10, Math.min(lensX, viewportWidth - lensSize - 10))
lensY = Math.max(10, Math.min(lensY, viewportHeight - lensSize - 10))

// 保存鼠标位置
this.mousePosition = {
  x: event.clientX,
  y: event.clientY
}

this.zoomPosition = { x, y }

// 创建放大效果
this.globalZoomStyle = {
  backgroundImage: `url(/images/artworks/${this.artwork.id}.jpg)`,
  backgroundSize: `${rect.width * this.zoomScale}px ${rect.height * this.zoomScale}px`,
  backgroundPosition: `${x * 100}% ${y * 100}%`,
  left: `${lensX}px`,
  top: `${lensY}px`,
  width: `${lensSize}px`,
```

```
    height: `${lensSize}px`,  
    opacity: 1,  
    transform: `translateZ(0)`,  
    zIndex: 9999  
  }  
},
```

3.2.6 个人画廊创建与编辑

- 功能描述：用户可以创建画廊，将作品加入画廊，并自由编排其位置、大小和旋转。
- 关键实现：
 1. 编辑器界面 (PersonalGallery.vue)：点击“创建”或“编辑”后，showEditor 变为 true，显示一个全屏模态框。该模态框分为三部分：左侧可用画作列表、中间的“工作区”、右侧的设置面板。
 2. 拖拽添加作品：可用画作列表中的每个作品都设置了 draggable="true"。startDrag 方法在拖拽开始时将作品信息存入 draggedItem。工作区 div 监听 drop 事件，在 dropArtwork 方法中，根据事件的 clientX 和 clientY 计算作品在工作区内的坐标，然后将一个包含位置信息的新对象添加到 currentGallery.items 数组中。
 3. 作品编辑：工作区内的每个作品项都监听了 mousedown 事件。startDragItem 用于拖拽移动位置，startResize 用于通过右下角手柄调整大小，这两个方法都通过动态添加和移除 mousemove 和 mouseup 的全局监听器来实现。

4. 保存画廊：saveGallery 方法是核心。它会校验画廊名称，然后将 currentGallery 的数据（名称、描述、隐私设置以及 items 数组中所有作品的位置和大小信息）整合成一个 galleryData 对象。根据 editingGallery 是否存在，调用 Vuex 的 updateGallery 或 createGallery action，最终通过 galleryAPI 将数据发送到后端。
5. 画廊列表展示：主页面通过 v-for 遍历从 Vuex 获取的 personalGalleries 数组，将每个画廊渲染成一个 .gallery-card。getActualGalleryItems 方法负责从每个画廊数据中提取前几幅作品的图片，用于生成卡片上的预览图。
6. 全屏编辑器：点击创建或编辑按钮，会设置 showEditor 为 true，弹出一个覆盖全屏的编辑器模态框。该编辑器采用三栏布局：
 - (1) 左栏（作品池）：v-for 渲染所有可用画作（availableArtworks），并为每个画作设置 draggable="true" 属性，通过 dragstart 事件记录被拖拽的作品信息。
 - (2) 中栏（工作区）：监听 drop 事件。dropArtwork 方法在事件触发时，根据鼠标位置计算出作品在工作区内的坐标，并将其添加至 currentGallery.items 数组。
 - (3) 右栏（设置面板）：包含画廊名称、描述、隐私、主题和布局的表单控件，均通过 v-model 与 currentGallery 对象双向绑定。
7. 自由布局与自动排列：工作区内的每个作品项都是可拖拽和缩放的。拖拽通过 startDragItem 方法实现；缩放则通过右下角的 resize-handle 和 startResize 方法实现。用户还可以点击“应用排列”按钮，触发 applyCurrentArrangement 方法，该方法会根据当前选择的布局模式（如

grid, spiral) 调用相应的排列函数，以编程方式重新计算并更新所有作品的位置。

8. 数据持久化：点击“保存画廊”时，saveGallery 方法会将 currentGallery 对象（包含所有作品的 ID 和其布局信息 x, y, width, height 等）打包，通过 galleryAPI 发送给后端。成功后，会重新调用 fetchMyGalleries action 来刷新本地的画廊列表，确保数据同步。

关键截图：

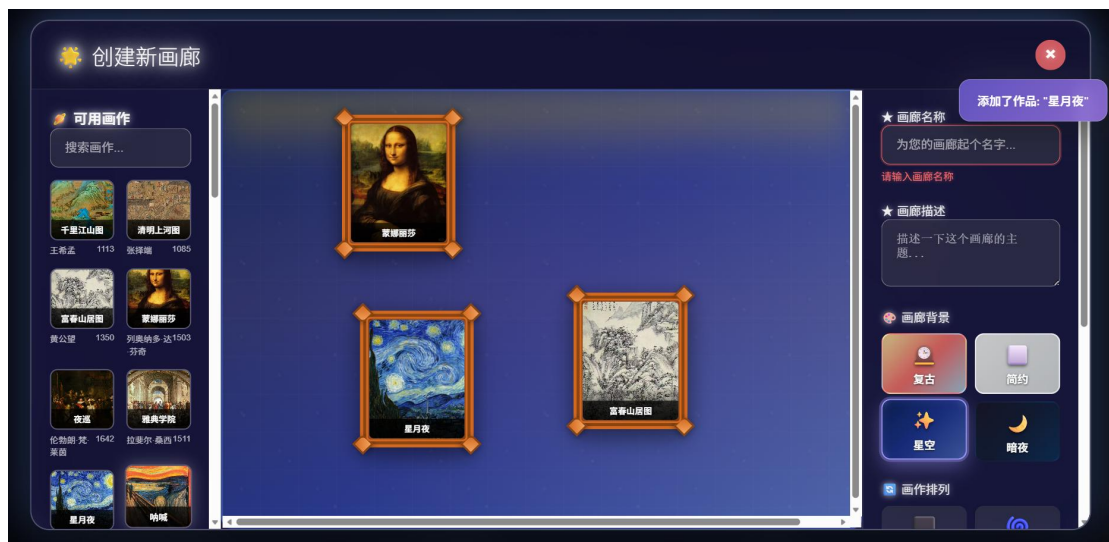


图 3.14 画廊编辑器



图 3.15 个人画廊列表

关键代码：

```
// 放置画作
const dropArtwork = (event) => {
  if (!draggedItem.value) {
    console.log('没有拖拽数据')
    return
  }

  console.log('放置作品:', draggedItem.value)

  const workspace = document.querySelector('.editor-workspace')
  if (!workspace) {
    console.error('找不到工作区')
    return
  }

  const rect = workspace.getBoundingClientRect()
  const x = event.clientX - rect.left - (draggedItem.value.width / 2)
  const y = event.clientY - rect.top - (draggedItem.value.height / 2)

  // 确保使用数据库中的作品ID
  const artworkId = draggedItem.value.id

  if (!artworkId) {
    console.error('无效的作品ID:', artworkId)
    showMessage('作品数据不完整, 无法添加', 'warning')
    return
  }
}
```

图 3.16 拖拽画作位置代码逻辑实现

保存画廊的代码实现：

```
// 保存画廊

const saveGallery = async () => {
  if (!currentGallery.value.name.trim()) {
    showMessage('请输入画廊名称', 'warning')
    return
  }

  try {
    console.log('=== 开始保存画廊 ===')
    console.log('编辑模式:', !!editingGallery.value)
    console.log('画廊名称:', currentGallery.value.name)
  }
}
```



```
console.log('作品数量:', currentGallery.value.items.length)
```

```
// 准备作品数据
```

```
const artworks = currentGallery.value.items.map(item => ({
  artwork_id: item.artwork_id || item.id,
  position_x: Math.round(item.x || 0),
  position_y: Math.round(item.y || 0),
  width: Math.round(item.width || 150),
  height: Math.round(item.height || 180),
  scale: item.scale || 1.0,
  rotation: item.rotate || 0
}))
```

```
if (artworks.length === 0) {
  showMessage('画廊中至少需要一个作品', 'warning')
  return
}
```

```
const galleryData = {
  name: currentGallery.value.name.trim(),
  description: currentGallery.value.description || '',
  privacy: currentGallery.value.privacy || 'public',
  theme: currentGallery.value.theme || 'starry',
  layout: currentGallery.value.layout || 'grid',
  artworks: artworks
}
```

```
console.log('发送画廊数据:', galleryData)
```

```
let result
```

```
if (editingGallery.value) {
  console.log('更新画廊:', editingGallery.value.id)
  result = await store.dispatch('updateGallery', {
    galleryId: editingGallery.value.id,
    galleryData: galleryData
  })
  showMessage('🌟 画廊已更新! ')
```

```

    } else {
      console.log('创建新画廊')
      result = await store.dispatch('createGallery', galleryData)
      showMessage('🌟 画廊创建成功! ')
    }

    console.log('保存结果:', result)

    // 刷新画廊列表
    await loadPersonalGalleries()

    showEditor.value = false
    editingGallery.value = null

  } catch (error) {
    console.error('保存画廊失败:', error)
    showMessage('保存画廊失败: ' + (error.message || '未知错误'), 'warning')
  }
}

```

3.2.4 社区系统与好友互动

- 功能描述：添加好友、实时聊天、查看和分享好友的画廊。
- 关键实现：
 1. 好友与聊天 (GalleryCommunity.vue)：页面挂载后，通过 loadCommunityData 方法并行请求好友列表、好友请求、通知和所有公开画廊。点击好友列表中的好友会触发 selectFriend 方法，该方法会更新 activeFriend 状态，并调用 loadConversation 从后端获取与该好友的聊天记录。发送消息时，sendMessage 方法调用 messageAPI.sendMessage，成功后将新消息添加到本地对话数组的头部。

2. 画廊分享：在个人画廊或社区画廊卡片上，点击分享按钮会触发 `showShareGalleryModal`。此模态框允许用户选择好友并发送。`shareGalleryToFriends` 方法调用 `messageSharesAPI.batchShareGallery`，将画廊 ID、好友 ID 列表和附加消息发送给后端。后端处理后，会在好友的聊天窗口中创建一条特殊的分享类型消息。
3. 权限控制：在 `GalleryCommunity.vue` 中，`canViewGallery` 方法会根据画廊的隐私设置（`public`, `friends`, `private`）和当前用户是否为作者的好友来判断是否可以查看。无权限的画廊卡片会显示一个锁定覆盖层。
4. 聚合数据加载：Mounted 时调用的 `loadCommunityData` 方法是页面初始化入口，它会并行地 dispatch 多个 Vuex action（`fetchFriends`, `fetchNotifications` 等），一次性获取页面所需的全部初始社交数据。
5. 好友聊天：点击好友列表中的一项会更新 `activeFriend` 状态，并触发 `loadConversation` 方法调用 `messageAPI` 获取与该好友的聊天记录。发送消息 `sendMessage` 则调用 `messageAPI.sendMessage`，并通过乐观更新的方式立即将消息显示在聊天窗口中。
6. 画廊展示与权限：`displayedGalleries` 计算属性负责对所有获取到的画廊进行筛选和排序。v-for 循环渲染时，会调用 `canViewGallery` 方法来判断当前用户是否有权查看该画廊。该方法会检查画廊的 `privacy` 属性，如果是 `friends-only`，则会进一步检查画廊作者 ID 是否存在于当前用户的好友 ID 列表中（`this.isFriend(gallery.user_id)`）。根据结果，决定是否显示一个锁定的覆盖层。

7. 模态框交互：添加好友、处理好友请求、分享画廊等复杂交互都通过独立的模态框实现。这些模态框的显示与否由 `showAddFriendModal` 等布尔值数据控制，并通过 `v-if` 指令在模板中条件性渲染。

关键截图：

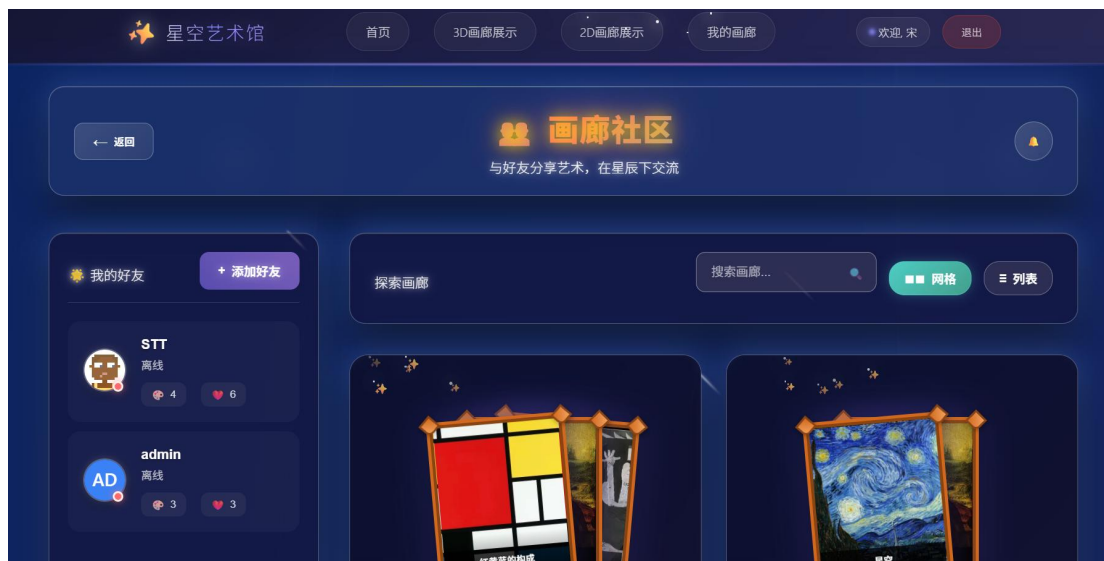


图 3.17 社区界面

发送好友申请的代码逻辑：

```
async sendFriendRequest(user) {
  console.log(' === 开始发送好友请求 ===')
  console.log('目标用户:', user)

  try {
    this.friendRequestResult = null
    // 使用 Vuex actions 发送好友请求
    await this.store.dispatch('sendFriendRequest', user.id)
    console.log('✔ store 调用成功')
    const userIndex = this.searchResults.findIndex(u => u.id === user.id)
    if (userIndex !== -1) {
      this.searchResults[userIndex].friend_status = 'request_sent'
    }
  }
  await this.loadFriendRequests()
}
```

```

    this.friendRequestResult = {
      type: 'success',
      message: `已向 ${user.username} 发送好友请求`
    }
    this.showMessage(`已向 ${user.username} 发送好友请求`, 'success')
    setTimeout(() => {
      this.friendRequestResult = null
    }, 3000)
  } catch (storeError) {
    console.error('✖ store 调用失败:', storeError)
    this.friendRequestResult = {
      type: 'error',
      message: storeError.response?.data?.error || '发送好友请求失败'
    }
    this.showMessage(`发送失败: ${storeError.response?.data?.error || storeError.message}`,
      'error')
    setTimeout(() => {
      this.friendRequestResult = null
    }, 3000)
  }
},

```

给好友发送信息的代码逻辑:

```

async sendMessage() {
  if (!this.newMessage.trim() || !this.activeFriend) {
    console.log('消息为空或没有选中好友')
    return
  }

  console.log('发送消息给:', this.activeFriend.username)

  try {
    const response = await messageAPI.sendMessage({
      receiver_id: this.activeFriend.id,
      content: this.newMessage.trim(),
      type: 'text'
    })

    if (response.success && response.message) {
      const message = response.message
    }
  }
}

```

```

        console.log('✔ 消息发送成功:', message)
        this.conversationMessages.unshift(message)
        this.newMessage = ''
        this.$nextTick(() => {
            this.scrollToBottom('chatMessages')
        })
        this.showMessage('消息发送成功', 'success')
    } else {
        console.error('消息发送失败:', response.error)
        this.showMessage(`发送失败: ${response.error}`, 'error')
    }
} catch (error) {
    console.error('发送消息失败:', error)
    const tempMessage = {
        id: Date.now(),
        sender_id: this.user?.id,
        receiver_id: this.activeFriend.id,
        content: this.newMessage,
        created_at: new Date().toISOString(),
        sender_name: this.user?.username
    }
    this.conversationMessages.unshift(tempMessage)
    this.newMessage = ''
    this.showMessage('消息已发送（本地模拟）', 'info')
    this.$nextTick(() => {
        this.scrollToBottom('chatMessages')
    })
}
},

```

3.2.8 画廊详情页 (GalleryDetail.vue)

- 功能描述：用于展示由其他用户创建的、具有自定义布局的个人画廊。
- 关键实现：

1. 严格的权限校验：此页面的核心在于权限控制。loadGalleryDetail 方法首先通过 galleryAPI.getGalleryDetail 获取画廊数据。后端返回的数据中包含一个关键字段 _canView。前端严格依赖此字段判断用户是否有权查看。如果

为 false，则页面会渲染一个友好的 permissionError 提示（例如，“这是好友专属画廊，请先添加好友”），而不是画廊内容。这种后端驱动的权限判断比纯前端逻辑更安全可靠。

2. 自定义布局渲染：在获取查看权限后，页面通过 v-for 遍历 gallery.artworks 数组。每个作品项的数据中都包含了作者在编辑器中保存的 position_x, position_y, width, height, scale 等布局信息。getArtworkStyle 方法将这些数据直接应用到每个作品 div 的 CSS style 上，从而在访客端精确地复现了画廊创建者设计的独特布局。
3. 互动与分享：页面提供了点赞画廊 (toggleLikeGallery) 和评论画廊 (submitComment) 的功能，这些功能直接调用 galleryAPI 中对应的接口。分享按钮 (shareGalleryLink) 则会复制当前画廊的 URL 到剪贴板，并弹出一个引导用户去社区分享的确认框。

关键截图：

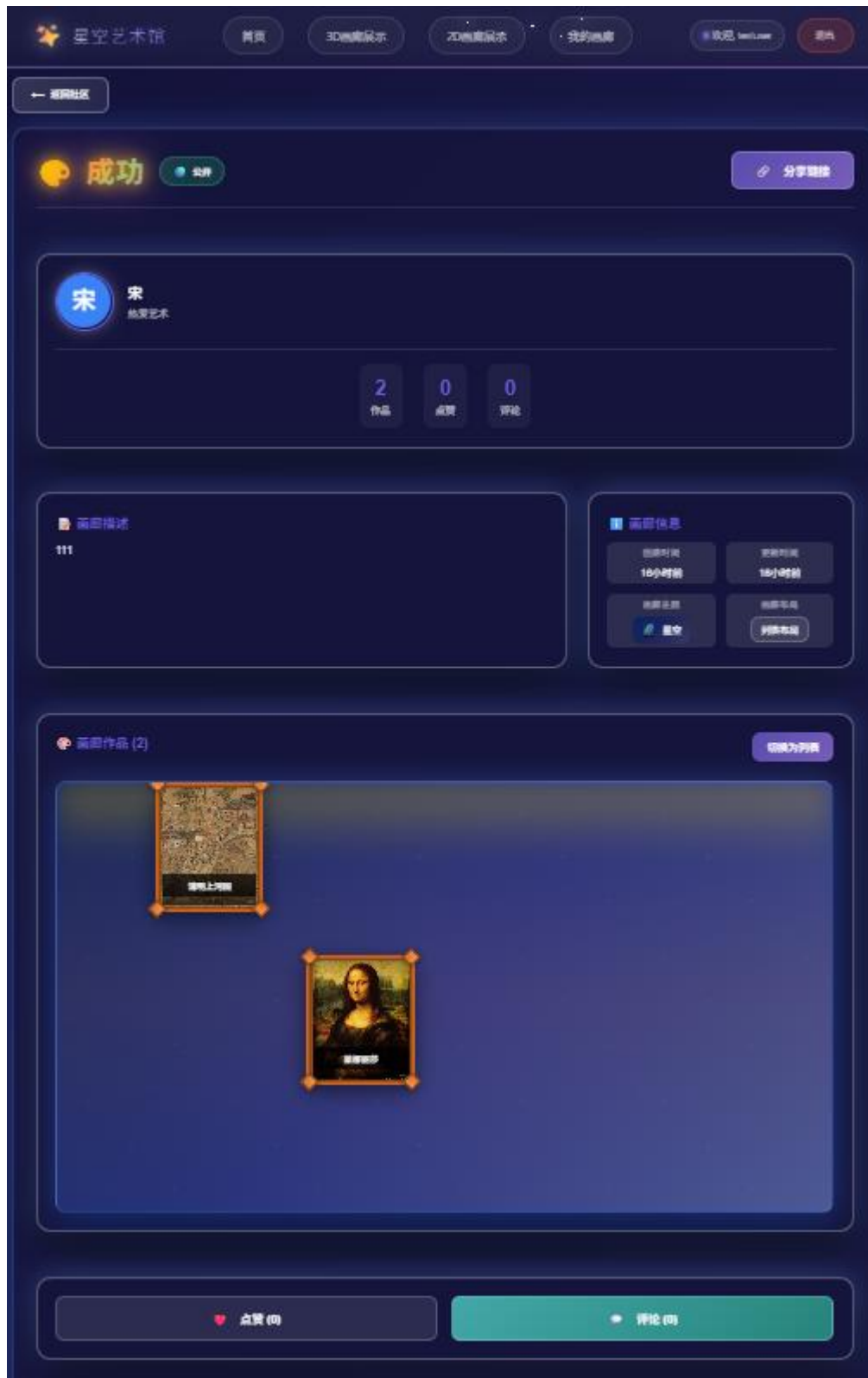


图 3.18 作品详情页

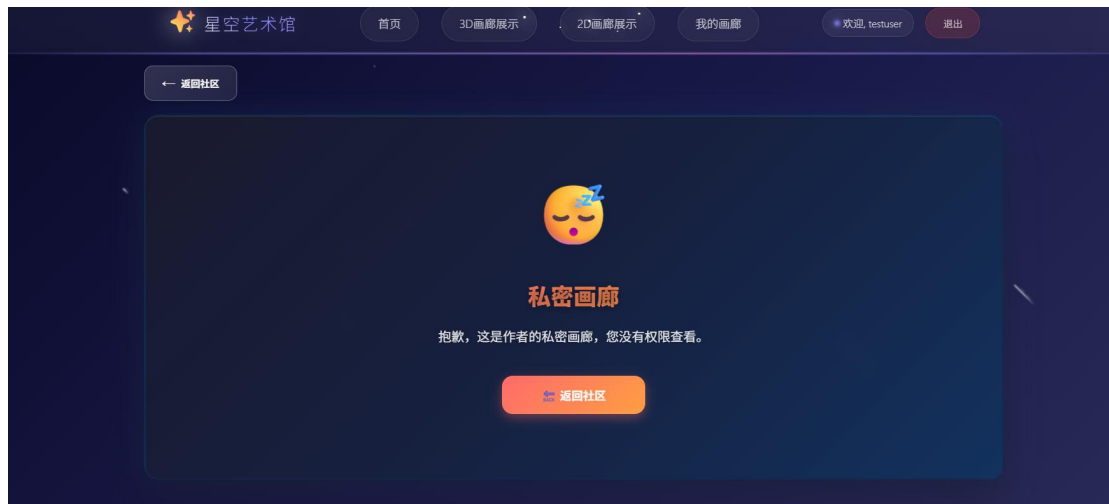


图 3.19 无权限的画廊提示

权限判断关键代码:

```
const loadGalleryDetail = async () => {
  isLoading.value = true
  permissionError.value = null

  try {
    console.log(`加载画廊详情: ${props.id}`)
    const response = await galleryAPI.getGalleryDetail(props.id)
    console.log('画廊详情 API 响应:', response)

    if (response.success && response.gallery) {
      gallery.value = response.gallery

      // 【关键】使用后端返回的权限字段 _canView
      const canView = gallery.value._canView !== undefined
        ? gallery.value._canView
        : canViewGallery.value

      console.log(`权限检查: _canView=${gallery.value._canView}, 前端计算=${canViewGallery.value}, 最终=${canView}`)

      if (!canView) {
        // 根据隐私设置显示不同的提示
        if (gallery.value.privacy === 'friends') {
          permissionError.value = {
```

icon: ' ',
title: '好友专属画廊',
message: `这是作者「``${gallery.value.username}``」的好友可见画廊。您需要成为作者的好友才能查看。`,

```
    actionText: '添加好友',  
    action: () => {  
      router.push({  
        path: '/gallery-community',  
        query: {  
          add_friend: gallery.value.user_id,  
          username: gallery.value.username  
        }  
      })  
    }  
  }  
} else if (gallery.value.privacy === 'private') {  
  permissionError.value = {  
    icon: ' ',  
    title: '私密画廊',  
    message: '抱歉，这是作者的私密画廊，您没有权限查看。',  
    actionText: '返回社区',  
    action: () => router.push('/gallery-community')  
  }  
} else {  
  permissionError.value = {  
    icon: ' ',  
    title: '没有权限',  
    message: '您没有权限查看此画廊。',  
    actionText: '返回社区',  
    action: () => router.push('/gallery-community')  
  }  
}  
return  
}
```

```
// 加载评论  
await loadGalleryComments()  
} else {  
  permissionError.value = {  
    icon: ' ',  
    title: '画廊不存在',  
    message: '您访问的画廊可能已被删除或链接无效。',  
    actionText: '返回社区',
```

```

        action: () => router.push('/gallery-community')
      }
    }
  } catch (error) {
    console.error('加载画廊详情失败:', error)

    if (error.response?.status === 404) {
      permissionError.value = {
        icon: ' ',
        title: '画廊不存在',
        message: '您访问的画廊可能已被删除或链接无效。',
        actionText: '返回社区',
        action: () => router.push('/gallery-community')
      }
    } else if (error.response?.status === 403) {
      permissionError.value = {
        icon: ' ',
        title: '没有权限',
        message: '您没有权限查看此画廊。',
        actionText: '返回社区',
        action: () => router.push('/gallery-community')
      }
    } else {
      permissionError.value = {
        icon: '✖',
        title: '加载失败',
        message: '加载画廊时发生错误, 请稍后再试。',
        actionText: '返回社区',
        action: () => router.push('/gallery-community')
      }
    }
  } finally {
    isLoading.value = false
  }
}

async fetchMyGalleries({ commit }) {
  try {
    // 1. 调用 API 获取数据
    const response = await galleryAPI.getMyGalleries()

    // 2. 使用 .map() 转换数据, 确保前端组件所需的所有字段都存在且格式正确
    const galleries = (response.galleries || []).map(gallery => ({
      id: gallery.id,

```

```

    name: gallery.name,
    description: gallery.description,
    privacy: gallery.privacy,
    theme: gallery.theme || 'starry', // 提供默认主题
    layout: gallery.layout || 'grid', // 提供默认布局
    artworks: gallery.artworks || [], // 确保 artworks 数组存在
    items: gallery.artworks || [], // 为了兼容您组件中可能存在的 items 用法
    createdAt: gallery.created_at || gallery.createdAt || new Date().toISOString(), // 兼容不同命名并提供备用

    updated_at: gallery.updated_at,
    likes_count: gallery.likes_count || 0,
    comments_count: gallery.comments_count || 0,
    is_liked: gallery.is_liked || false,
    user_id: gallery.user_id,
    author: gallery.author || { // 确保 author 对象存在
      id: gallery.user_id,
      name: gallery.username,
      avatar: gallery.author_avatar
    }
  }
}))

    commit('SET_MY_GALLERIES', galleries)

  console.log('✔ 获取个人画廊成功, 已存入 state:', galleries.length, '个')
  return galleries
} catch (error) {
  // 4. 错误处理: 如果 API 失败, 则从本地存储加载数据
  console.error('✖ 获取个人画廊失败, 尝试从本地恢复:', error)

  const localGalleries = JSON.parse(localStorage.getItem('galleries')) || []

  // 即使失败, 也要调用 SET_MY_GALLERIES 来保证 state 数据结构的正确性
  commit('SET_MY_GALLERIES', localGalleries)

  return localGalleries
}
},

```

四、系统测试

4.1 测试用例

以下是测试用例的一个实例：

表 4.1 登录注册功能测试用例

编号	模块	测试步骤	预期结果	实际结果
1	登录注册	输入错误的账号或密码	提示登录失败	与预期结果一致
2		注册的账号为空	提示账号不能为空	与预期结果一致
3		注册的账号已存在	提示账号已存在	与预期结果一致
4		输入已存在的用户名。	提示“用户名已存在”。	与预期结果一致

表 4.2 2D 画廊浏览功能测试用例

编号	模块	测试步骤	预期结果	实际结果
1	2D 画廊浏览	在画廊页面按住鼠标左键并拖拽。	视图跟随鼠标平移。	与预期结果一致
2		在画廊页面滚动鼠标滚轮。	视图以鼠标指针为中心进行缩放。	与预期结果一致

表 4.3 画廊详情功能测试用例

编号	模块	测试步骤	预期结果	实际结果
1	画作详情	点击任意画作。	跳转到该画作的详情页，显示正确信息，并开始播放背景音乐。	与预期结果一致

表 4.4 放大镜功能测试用例

编号	模块	测试步骤	预期结果	实际结果
1	放大镜功能	在画作详情页，鼠标悬停在图片上。	出现一个跟随鼠标的圆形放大镜，显示局部放大图像。	与预期结果一致

表 4.5 个人画廊创建功能测试用例

编号	模块	测试步骤	预期结果	实际结果
1	个人画廊创建	登录后，在个人画廊页面点击“创建画廊”，填写名称，添加至少一幅作品后保存。	创建成功，页面返回画廊列表并显示新创建的画廊卡片。	与预期结果一致
2		在工作区内拖拽移动一个作品，然后保存。	保存成功，再次进入编辑时，作品位置为上次保存的位置。	与预期结果一致

表 4.6 个人画廊删除功能测试用例

编号	模块	测试步骤	预期结果	实际结果
1	个人画廊删除	点击画廊卡片上的删除按钮，并在弹出的确认框中点击确认。	画廊被成功删除，并在列表中消失。	与预期结果一致

表 4.7 社区好友功能测试用例

编号	模块	测试步骤	预期结果	实际结果
1	登录注册	在社区搜索一个存在的用户并发送好友请求。	对方用户会收到一条好友请求通知。	与预期结果一致

表 4.8 登录注册功能测试用例

编号	模块	测试步骤	预期结果	实际结果
----	----	------	------	------

1	社区聊天	与好友打开聊天窗口并发送一条消息。	消息出现在聊天记录中，对方能接收到。	与预期结果一致
---	------	-------------------	--------------------	---------

五、总结

通过本次虚拟艺术画廊展示系统的课程设计，我收获颇丰。

首先，在技术层面，这是我第一次系统性地、独立地完成一个功能较为复杂的 Vue 单页应用。通过这个项目，我深入实践了 Vue 的组件化思想、API 的使用，以及 Vue Router 和 Vuex 在真实项目中的应用模式。特别是个人画廊编辑器中复杂的拖拽、缩放、布局等交互功能的实现，锻炼了我解决复杂前端逻辑问题的能力。社区模块的开发则让我对客户端与服务器的数据交互、状态同步有了更深刻的理解。

其次，在项目管理和设计层面，从需求分析、系统设计到具体实现和测试，我完整地走了一遍软件开发的流程。学会了如何将一个抽象的需求分解为具体的功能模块，如何设计数据结构（如画廊和作品的数据库模型），以及如何规划组件之间的关系。这个过程让我认识到，清晰的设计是高效开发的基石。

当然，在开发过程中也遇到了不少挑战。例如，2D 画廊的性能优化问题，最初大量的 DOM 元素导致拖拽卡顿，后来通过优化渲染逻辑和事件节流得以改善。又如，社区聊天功能的实时性问题，虽然目前是通过轮询或手动刷新实现的，但让我认识到了 WebSocket 等实时通信技术的重要性，这也是我未来希望学习和改进的方向。

总而言之，这次课程设计不仅是对我前端技术能力的一次全面检验和提升，更是一次宝贵的独立项目开发经验。它让我体会到了从零到一创造一个产品的乐趣与

挑战，也为我未来的学习和职业发展奠定了坚实的基础。